

Système Multi-Agents Médical

avec LangGraph

Rapport Technique

Réalisé par :

Aya Aguilou

Encadré par :

Pr. Mohamed YOUSSEFI

Année Universitaire 2025-2026

Table des matières

1	Introduction	3
1.1	Contexte du Projet	3
1.2	Objectifs Pédagogiques	3
1.3	Cadre Éthique	3
2	Architecture du Système	4
2.1	Architecture Globale	4
2.2	Structure du Projet	4
2.3	Choix Technologiques	4
3	Workflow LangGraph	5
3.1	Schéma du Workflow	5
3.2	Description des Nœuds	5
3.3	État Partagé (State)	5
3.4	Transitions	5
4	Agents et Tools	7
4.1	Agent Supervisor	7
4.2	Agent Diagnostic	7
4.3	Outils (Tools)	7
4.3.1	Patient Tools	7
4.3.2	Care Tools	7
4.4	Agent Physician Review (Human-in-the-Loop)	8
4.5	Agent Report	8
5	Human-in-the-Loop	9
5.1	Principe	9
5.2	Implémentation	9
5.3	Interface Médecin	9
6	API FastAPI	10
6.1	Endpoints	10
6.2	Exemple d'utilisation	10
7	Intégration MCP	11
7.1	Principe	11
7.2	Tools MCP Implémentés	11
7.3	Exemple d'utilisation	11
8	Interface Utilisateur	12
8.1	Écrans	12
8.2	Flux Utilisateur	12
8.3	Captures d'écran	12
9	Tests et Cas Cliniques	14
9.1	Cas 1 : Syndrome Respiratoire Simple	14
9.2	Cas 2 : Cas avec Red Flags	14
9.3	Cas 3 : Cas Bénin	14

10 Difficultés Rencontrées	15
10.1 Déploiement de l'API	15
10.2 Intégration Ollama	15
10.3 Problèmes de CORS	15
10.4 Boucle infinie dans LangGraph	15
10.5 Solution adoptée	15
11 Conclusion	16
11.1 Bilan	16
11.2 Compétences Acquises	16
11.3 Perspectives d'Amélioration	16
11.4 Mention Légale	16
12 Annexes	17
12.1 Liens utiles	17
12.2 Commandes utiles	17
12.3 Configuration requise	17
12.4 Dépendances	17

1 Introduction

1.1 Contexte du Projet

Ce rapport présente la conception et la réalisation d'un système multi-agents médical utilisant **LangGraph**. Le projet s'inscrit dans le cadre d'un exercice académique visant à simuler un workflow d'orientation clinique.

1.2 Objectifs Pédagogiques

- Modéliser un workflow multi-agents avec LangGraph
- Gérer un état partagé entre plusieurs agents
- Utiliser des outils (tools) et intégrer le Human-in-the-Loop
- Exposer le graphe via une API FastAPI
- Intégrer MCP (Model Context Protocol)
- Développer une interface utilisateur avec Streamlit
- Tester et déboguer le graphe dans LangGraph Studio

1.3 Cadre Éthique

Le projet est un exercice académique. Le système ne doit pas être présenté comme un dispositif médical. Les termes recommandés sont : *orientation clinique préliminaire*, *synthèse clinique* et *recommandation intermédiaire*.

Le rapport final mentionne explicitement : « **Ce système ne remplace pas une consultation médicale.** »

2 Architecture du Système

2.1 Architecture Globale

L'application suit une architecture en couches :

Couche	Technologie
Interface Utilisateur	Streamlit
API	FastAPI
Workflow	LangGraph
Agents	LangChain
Persistance	MemorySaver
MCP	FastMCP

FIGURE 1 – Architecture technologique

2.2 Structure du Projet

Dossier/Fichier	Description
backend/app/api.py	API FastAPI
backend/app/graph.py	Workflow LangGraph
backend/app/state.py	État partagé
backend/app/nodes/supervisor.py	Agent superviseur
backend/app/nodes/diagnostic_agent.py	Agent diagnostic (5 questions)
backend/app/nodes/physician_review.py	Human-in-the-Loop (médecin)
backend/app/nodes/report_agent.py	Agent rapport final
backend/app/tools/patient_tools.py	Outils patient
backend/app/tools/care_tools.py	Outils soins
frontend/app.py	Interface Streamlit
mcp_server/server.py	Serveur MCP
requirements.txt	Dépendances

TABLE 1 – Structure des fichiers du projet

2.3 Choix Technologiques

- **LangGraph** : Framework pour créer des workflows d'agents avec des états persistants et des transitions conditionnelles.
- **FastAPI** : Framework web rapide pour créer l'API REST.
- **Streamlit** : Framework pour créer des interfaces utilisateur en Python.
- **MCP** : Protocole pour connecter des outils externes aux agents.

3 Workflow LangGraph

3.1 Schéma du Workflow

Le workflow minimal attendu est :

```
START → Supervisor → DiagnosticAgent
  → Tool: ask patient (5 questions)
  → Tool: recommend interim care
  → Supervisor → PhysicianReview (Human-in-the-Loop)
  → Supervisor → ReportAgent → END
```

3.2 Description des Nœuds

Nœud	Rôle
Supervisor	Orchestre le workflow et décide de l'étape suivante en fonction de l'état
Diagnostic Agent	Pose 5 questions au patient, analyse les réponses et produit une synthèse clinique préliminaire
Physician Review	Étape Human-in-the-Loop où le médecin valide le dossier
Report Agent	Génère le rapport final structuré

TABLE 2 – Nœuds du workflow LangGraph

3.3 État Partagé (State)

L'état partagé est défini comme suit :

```
1 class MedicalState(TypedDict, total=False):
2     messages: Annotated[list, add_messages]
3     next: Literal["diagnostic_agent", "physician_review",
4                  "report_agent", "FINISH"]
5     patient_id: str
6     initial_complaint: str
7     question_count: int
8     patient_answers: list
9     diagnostic_summary: str
10    interim_care: str
11    physician_treatment: str
12    final_report: str
```

3.4 Transitions

Les transitions entre les nœuds sont gérées par le Supervisor :

1. **Supervisor → DiagnosticAgent** : Pour poser les questions
2. **DiagnosticAgent → Supervisor** : Retour après chaque question
3. **Supervisor → PhysicianReview** : Après les 5 questions
4. **PhysicianReview → Supervisor** : Après validation du médecin

5. **Supervisor** → **ReportAgent** : Pour générer le rapport
6. **ReportAgent** → **END** : Fin du workflow

4 Agents et Tools

4.1 Agent Supervisor

Le superviseur orchestre le workflow en décidant de l'étape suivante. Sa logique est la suivante :

```
1 def supervisor_node(state):
2     if state.get("final_report"):
3         return {"next": "FINISH"}
4     if state.get("physician_treatment"):
5         return {"next": "report_agent"}
6     if state.get("diagnostic_summary"):
7         return {"next": "physician_review"}
8     if state.get("question_count", 0) >= 5:
9         return {"next": "diagnostic_agent"}
10    return {"next": "diagnostic_agent"}
```

4.2 Agent Diagnostic

L'agent diagnostic pose 5 questions successives au patient :

1. "Depuis combien de temps avez-vous ces symptômes?"
2. "Avez-vous de la fièvre ? Si oui, quelle température?"
3. "Avez-vous des difficultés à respirer?"
4. "Avez-vous des douleurs ? Si oui, où et comment?"
5. "Avez-vous d'autres symptômes à mentionner?"

4.3 Outils (Tools)

4.3.1 Patient Tools

```
1 def ask_patient(question_index: int) -> str:
2     questions = [
3         "Depuis combien de temps avez-vous ces sympt mes ?",
4         "Avez-vous de la fi vre ? Si oui, quelle temp rature ?",
5         "Avez-vous des difficult s respirer ?",
6         "Avez-vous des douleurs ? Si oui, o et comment ?",
7         "Avez-vous d'autres sympt mes mentionner ?"
8     ]
9     return questions[question_index]
```

4.3.2 Care Tools

```
1 def recommend_interim_care(symptoms: str) -> str:
2     if "fi vre" in symptoms.lower():
3         return "Repos complet, hydratation abondante, parac tamol si
4             fi vre > 38.5 C "
5     elif "douleur" in symptoms.lower():
6         return "Repos, application de glace, antalgiques simples"
7     else:
8         return "Repos et hydratation. Consulter si aggravation."
```


4.4 Agent Physician Review (Human-in-the-Loop)

Le médecin reçoit la synthèse clinique et la recommandation intermédiaire. Il peut :

- Approuver le rapport
- Ajouter un commentaire médical
- Proposer un traitement ou une conduite à tenir

4.5 Agent Report

L'agent report génère le rapport final structuré contenant :

- L'ID du patient
- La plainte initiale
- La synthèse clinique
- La recommandation intermédiaire
- Le traitement prescrit par le médecin
- La mention légale

5 Human-in-the-Loop

5.1 Principe

Le Human-in-the-Loop est une étape où le workflow s'interrompt pour permettre à un humain (le médecin) d'intervenir.

5.2 Implémentation

Dans LangGraph, cette interruption est gérée via la fonction `physician_review_node` qui vérifie si le médecin

```
1 def physician_review_node(state):  
2     if state.get("physician_treatment"):  
3         state["next"] = "supervisor"  
4     else:  
5         state["pending_doctor"] = True  
6         state["next"] = "physician_review"  
7     return state
```

5.3 Interface Médecin

L'interface permet au médecin de :

1. Voir les réponses du patient
2. Ajouter son avis médical
3. Approuver le rapport
4. Générer le rapport final

6 API FastAPI

6.1 Endpoints

Méthode	Endpoint	Description
POST	/consultation/start	Démarrer une consultation
POST	/consultation/resume	Reprendre une consultation
GET	/consultation/{thread_id}	Récupérer l'état
GET	/consultation/{thread_id}/report	Récupérer le rapport
POST	/doctor/{session_id}	Validation du médecin
GET	/mcp/advice/{symptom}	Conseils MCP
GET	/mcp/redflags/{symptom}	Alertes MCP
GET	/mcp/medication/{condition}	Médicaments MCP

TABLE 3 – Endpoints de l'API FastAPI

6.2 Exemple d'utilisation

```
1 # Démarrer une consultation
2 POST /consultation/start
3 Body: {"patient_id": "P001", "initial_complaint": "Toux"}
4
5 # Répondre à une question
6 POST /consultation/resume
7 Body: {"thread_id": "1", "answer": "Depuis 3 jours"}
8
9 # Récupérer le rapport
10 GET /consultation/1/report
```

7 Intégration MCP

7.1 Principe

MCP (Model Context Protocol) permet d'intégrer des outils externes aux agents.

7.2 Tools MCP Implémentés

Outil	Description
get_medical_advice	Donne des conseils médicaux généraux
check_red_flags	Vérifie les signes d'alerte
get_medication	Recommande des médicaments courants

TABLE 4 – Tools MCP

7.3 Exemple d'utilisation

```
1 # Conseil pour la fièvre
2 GET /mcp/advice/fièvre
3 Reponse: "Repos complet, hydratation abondante, paracétamol si > 38.5°C"
4
5 # Vérification des red flags
6 GET /mcp/redflags/douleur thoracique
7 Reponse: "ALERTE: douleur thoracique détectée"
```

8 Interface Utilisateur

8.1 Écrans

Écran	Contenu
Écran 1 : Saisie	Saisie du cas initial patient
Écran 2 : Questions	5 questions/réponses patient
Écran 3 : Médecin	Revue médicale avec validation
Écran 4 : Rapport	Rapport final et téléchargement

TABLE 5 – Écrans de l'interface Streamlit

8.2 Flux Utilisateur

1. Le patient clique sur "Démarrer la consultation"
2. L'agent diagnostic pose 5 questions
3. Le patient répond à chaque question
4. Une synthèse clinique est générée
5. Le médecin reçoit la synthèse
6. Le médecin valide et ajoute un commentaire
7. Le rapport final est généré
8. Le rapport peut être téléchargé

8.3 Captures d'écran



FIGURE 2 – Écran 1 : Page d'accueil de l'application

Deploy



Consultation Médicale IA

Système multi-agents pour l'orientation clinique préliminaire

Question 1 / 5



1. Depuis combien de temps avez-vous ces symptômes ?

Votre réponse

Écrivez votre réponse ici...

Envoyer

FIGURE 3 – Écran 2 : Questions posées au patient

Deploy



Consultation Médicale IA

Système multi-agents pour l'orientation clinique préliminaire



Revue Médicale

Le médecin doit valider le dossier avant la génération du rapport final.

> Voir les réponses du patient

Avis du médecin

fièvre et grippe

☒
☒ J'approuve le rapport

FIGURE 4 – Écran 3 : Revue médicale (Human-in-the-Loop)

Deploy



Consultation Médicale IA

Système multi-agents pour l'orientation clinique préliminaire



Rapport Final

Rapport

RAPPORT FINAL

Synthèse basée sur 5 réponses.

Réponses du patient:

- depuis 3 jours
- oui 40 degrees
- non
- mal a la tete
- non

Avis du médecin: je pense que vous avez de la fièvre et la grippe

Validation: ☒ Approuvé

FIGURE 5 – Écran 4 : Rapport final

9 Tests et Cas Cliniques

9.1 Cas 1 : Syndrome Respiratoire Simple

Plainte : Toux et fièvre depuis 3 jours

Question	Réponse
1. Depuis combien de temps ?	Depuis 3 jours
2. Fièvre ?	Oui, 38.5°C
3. Difficultés respiratoires ?	Non
4. Douleurs ?	Mal de tête
5. Autres symptômes ?	Non

Recommandation intermédiaire : Repos, hydratation, paracétamol si fièvre > 38.5°C.

Avis du médecin : Repos, paracétamol 500mg.

9.2 Cas 2 : Cas avec Red Flags

Plainte : Douleur thoracique soudaine, essoufflement

Recommandation : Consulter immédiatement un médecin ou les urgences.

9.3 Cas 3 : Cas Bénin

Plainte : Petite coupure au doigt

Recommandation : Nettoyage, désinfection, pansement.

10 Difficultés Rencontrées

10.1 Déploiement de l'API

Le déploiement de l'API FastAPI a rencontré des problèmes de CORS et de connexion entre le frontend et le backend.

10.2 Intégration Ollama

L'intégration du LLM local avec Ollama a été complexe en raison de problèmes d'installation et de configuration sur Windows.

10.3 Problèmes de CORS

Le frontend Streamlit n'arrivait pas à communiquer avec l'API FastAPI. La solution a été d'utiliser une application Streamlit autonome.

10.4 Boucle infinie dans LangGraph

Le graphe entrait en boucle infinie à cause d'une mauvaise gestion des transitions. La solution a été de simplifier la logique du routeur.

10.5 Solution adoptée

Pour garantir le bon fonctionnement de l'application, une version autonome avec Streamlit a été développée, intégrant un simulateur d'API.

11 Conclusion

11.1 Bilan

Ce projet a permis de :

- Concevoir un workflow multi-agents avec LangGraph
- Implémenter un Human-in-the-Loop pour la validation médicale
- Développer une interface utilisateur intuitive
- Intégrer des outils MCP
- Tester le système sur 3 cas cliniques

11.2 Compétences Acquises

- Maîtrise de LangGraph et LangChain
- Développement d'API avec FastAPI
- Création d'interfaces avec Streamlit
- Gestion d'état et de workflow
- Intégration de Human-in-the-Loop

11.3 Perspectives d'Amélioration

- Intégration d'un LLM réel (Ollama, OpenAI)
- Persistance en base de données
- Export PDF du rapport
- Dockerisation de l'application
- Tests unitaires et d'intégration
- Historique des consultations
- Authentification et sécurité

11.4 Mention Légale

Ce système ne remplace pas une consultation médicale.

12 Annexes

12.1 Liens utiles

- GitHub : https://github.com/AyaAguilou/medical_multi_agents
- Streamlit Cloud : <https://medicalmultiaqents-cqhxcfmfm7jgnymyfuyyc.streamlit.app>
- Documentation LangGraph : <https://langchain-ai.github.io/langgraph/>
- Documentation FastAPI : <https://fastapi.tiangolo.com/>
- Documentation Streamlit : <https://docs.streamlit.io/>

12.2 Commandes utiles

```
1 # Lancer l'API
2 uvicorn backend.app.api:api --reload
3
4 # Lancer Streamlit
5 streamlit run frontend/app.py
6
7 # Installer les dépendances
8 pip install -r requirements.txt
```

12.3 Configuration requise

- Python 3.11 ou supérieur
- Pip
- Navigateur web

12.4 Dépendances

```
1 fastapi
2 uvicorn
3 streamlit
4 requests
5 langgraph
6 langchain
7 pydantic
8 python-dotenv
9 httpx
```